

BEC305 Python Programming — Module 3 Quiz

Pattern Matching with Regular Expressions and File Handling

Field	Value
Subject code	BEC305
Module	3 — Pattern Matching with Regular Expressions and File Handling
Duration	45 minutes
Maximum marks	50 (Part A 20 + Part B 30)
Attempts allowed	1
Passing marks	20
Textbook	Al Sweigart, Automate the Boring Stuff with Python, 1st ed., 2015

Instructions

- Part A carries 20 one-mark multiple choice questions. Exactly one option is correct.
- Part B carries 13 short questions totalling 30 marks.
- Write every regular expression as a raw string.
- State clearly whether a pattern is anchored and why.
- No computers or interpreters are permitted during the quiz.

Course outcome and Bloom coverage

Band	Questions	Share	Target
L1–L2 Remember / Understand	8	24%	~30%
L3–L4 Apply / Analyse	23	70%	~45%
L5–L6 Evaluate / Create	2	6%	~25%

Course outcome spread: CO3: 33. Total questions: 33.

Part A — Multiple choice (20 × 1 mark = 20 marks)

A1. Why should regular expressions be written as raw strings? [CO3 | L2 | 1 mark | medium | Ch.7, pp.150-152]

- (A) So backslashes reach the regex engine instead of being consumed as Python escape characters
- (B) Because raw strings are faster
- (C) Because re.compile rejects ordinary strings
- (D) Because raw strings use less memory

A2. What does search() return when the pattern is not found? [CO3 | L2 | 1 mark | easy | Ch.7, pp.150-152]

- (A) None

- (B) An empty Match object
- (C) An empty string
- (D) It raises an exception

A3. In a Match object, what does group(0) return? [C03 | L2 | 1 mark | medium | Ch.7, pp.152-153]

- (A) The entire match
- (B) The first capturing group
- (C) None
- (D) A tuple of all groups

A4. What does re.compile(r'(Bat|Batman)').search('Batman').group() return? [C03 | L4 | 1 mark | hard | Ch.7, pp.153-154]

- (A) 'Bat'
- (B) 'Batman'
- (C) None
- (D) A tuple

A5. For r'Bat(wo)?man' matching 'Batman', what is mo.group(1)? [C03 | L4 | 1 mark | hard | Ch.7, pp.154-155]

- (A) None
- (B) ''
- (C) 'wo'
- (D) IndexError

A6. What is the difference between * and + as quantifiers? [C03 | L2 | 1 mark | easy | Ch.7, pp.155-156]

- (A) * allows zero occurrences, + requires at least one
- (B) * is greedy and + is lazy
- (C) + allows zero occurrences, * requires at least one
- (D) They are identical

A7. What does r'<.*>' match in '<to serve><man> for dinner'? [C03 | L4 | 1 mark | hard | Ch.7, pp.156-157]

- (A) '<to serve><man>'
- (B) '<to serve>'
- (C) '<man>'
- (D) Nothing

A8. How do you make a quantifier nongreedy? [C03 | L2 | 1 mark | medium | Ch.7, pp.156-157]

- (A) Follow it with a question mark
- (B) Precede it with a caret
- (C) Wrap it in square brackets
- (D) Pass re.NONGREEDY

A9. What does findall() return when the pattern contains two capturing groups? [C03 | L4 | 1 mark | hard | Ch.7, pp.157-158]

- (A) A list of tuples, one tuple per match
- (B) A list of strings
- (C) A Match object

- (D) A dictionary

A10. How do you group part of a pattern without capturing it? [C03 | L3 | 1 mark | hard | Ch.7, pp.157-158]

- (A) (?:...)
- (B) (...)
- (C) [...]
- (D) {...}

A11. What does `\w` match? [C03 | L1 | 1 mark | easy | Ch.7, pp.158-159]

- (A) Any letter, digit or underscore
- (B) Any whitespace character
- (C) Any word of at least one letter
- (D) Any non-digit

A12. Inside square brackets, what does a leading caret mean? [C03 | L3 | 1 mark | medium | Ch.7, pp.158-162]

- (A) It negates the character class
- (B) It anchors to the start of the string
- (C) It is a literal caret
- (D) It makes the class case-insensitive

A13. Which pattern validates that a whole string is digits and nothing else? [C03 | L4 | 1 mark | hard | Ch.7, pp.159-162]

- (A) `r'^\d+$'`
- (B) `r'\d+'`
- (C) `r'^\d+'`
- (D) `r'\d*'`

A14. What does the dot match by default? [C03 | L2 | 1 mark | medium | Ch.7, pp.159-162]

- (A) Any single character except a newline
- (B) Any single character including a newline
- (C) A literal full stop only
- (D) Any sequence of characters

A15. Which flag lets you lay a long pattern out over several lines with comments? [C03 | L3 | 1 mark | medium | Ch.7, pp.162-165]

- (A) `re.VERBOSE`
- (B) `re.DOTALL`
- (C) `re.IGNORECASE`
- (D) `re.MULTILINE`

A16. Why should paths be built with `os.path.join()` rather than string concatenation? [C03 | L3 | 1 mark | medium | Ch.8, pp.173-177]

- (A) `join` inserts the correct separator for whichever operating system is running
- (B) `join` is faster than concatenation
- (C) `join` verifies that the file exists
- (D) Concatenation is a `SyntaxError` for paths

A17. What happens to an existing file when it is opened in 'w' mode? [C03 | L2 | 1 mark | medium | Ch.8, pp.180-184]

- (A) It is truncated to nothing immediately
- (B) The new data is appended to the end
- (C) An exception is raised
- (D) The file is opened read-only

A18. Why is the with statement preferred when opening a file? [C03 | L3 | 1 mark | medium | Ch.8, pp.180-184]

- (A) It closes the file even if an exception is raised inside the block
- (B) It opens the file faster
- (C) It allows more than one mode at a time
- (D) It is required by Python 3

A19. What is the main advantage of shelve over writing a text file by hand? [C03 | L4 | 1 mark | medium | Ch.8, pp.184-185]

- (A) Values come back as the same Python types, with no parsing needed
- (B) The file can be edited in any text editor
- (C) It is human-readable
- (D) It works without importing a module

A20. Why must a pformat file contain an assignment such as cats = [...] rather than the bare structure? [C03 | L5 | 1 mark | hard | Ch.8, pp.185-192]

- (A) Importing the module must bind a name you can then read
- (B) pformat cannot produce a bare structure
- (C) Python refuses to import a file without an assignment
- (D) The assignment makes the file smaller

Part B — Descriptive (13 questions, 30 marks)

B1. Explain the argument for regular expressions by contrasting them with a hand-written matcher. [C03 | L4 | 3 marks | Ch.7, pp.148-150]

B2. Explain greedy versus nongreedy matching with a worked example, and say why single-occurrence testing hides the difference. [C03 | L4 | 3 marks | Ch.7, pp.156-157]

B3. Explain how the return shape of findall() depends on the pattern, and how to keep it flat. [C03 | L4 | 3 marks | Ch.7, pp.157-158]

B4. Explain the difference between searching and validating, and write a pattern for each. [C03 | L4 | 3 marks | Ch.7, pp.159-162]

B5. Why must regular expressions be written as raw strings? [C03 | L3 | 2 marks | Ch.7, pp.150-152]

B6. Explain why alternation order matters, with an example. [C03 | L4 | 2 marks | Ch.7, pp.153-154]

B7. What does group(n) return for an optional group that did not match, and why does it matter? [C03 | L4 | 2 marks | Ch.7, pp.154-155]

B8. Explain what re.VERBOSE is for and when you would reach for it. [C03 | L4 | 2 marks | Ch.7, pp.162-165]

B9. Write a sub() call that redacts names after the word Agent, keeping only the initial, and explain how the backreference works. [C03 | L4 | 2 marks | Ch.7, pp.162-165]

B10. Explain why os.path.join should be used and what goes wrong without it. [C03 | L3 | 2 marks | Ch.8, pp.173-177]

B11. Write the code to total the sizes of all files in a folder and explain the check it needs. [C03 | L3 | 2 marks | Ch.8, pp.177-180]

B12. Contrast the three file modes r, w and a, and say which is dangerous and why. [C03 | L4 | 2 marks | Ch.8, pp.180-184]

B13. Compare shelve with pprint.pformat as ways of persisting a Python structure. [C03 | L5 | 2 marks | Ch.8, pp.184-192]

Answer key — Part A

Q	Answer	CO	Bloom	Reference
A1	A	CO3	L2	Ch.7, pp.150-152
A2	A	CO3	L2	Ch.7, pp.150-152
A3	A	CO3	L2	Ch.7, pp.152-153
A4	A	CO3	L4	Ch.7, pp.153-154
A5	A	CO3	L4	Ch.7, pp.154-155
A6	A	CO3	L2	Ch.7, pp.155-156
A7	A	CO3	L4	Ch.7, pp.156-157
A8	A	CO3	L2	Ch.7, pp.156-157
A9	A	CO3	L4	Ch.7, pp.157-158
A10	A	CO3	L3	Ch.7, pp.157-158
A11	A	CO3	L1	Ch.7, pp.158-159
A12	A	CO3	L3	Ch.7, pp.158-162
A13	A	CO3	L4	Ch.7, pp.159-162
A14	A	CO3	L2	Ch.7, pp.159-162
A15	A	CO3	L3	Ch.7, pp.162-165
A16	A	CO3	L3	Ch.8, pp.173-177
A17	A	CO3	L2	Ch.8, pp.180-184
A18	A	CO3	L3	Ch.8, pp.180-184
A19	A	CO3	L4	Ch.8, pp.184-185
A20	A	CO3	L5	Ch.8, pp.185-192

Part A — why each option is right or wrong

A1. Correct answer: (A) So backslashes reach the regex engine instead of being consumed as Python escape characters

- (A) Correct - without the `r` prefix, Python processes the escapes first and the engine never sees the backslashes you wrote.
- (B) Wrong - there is no speed difference; the `r` prefix only changes how the literal is parsed.
- (C) Wrong - `re.compile` accepts either, which is why the bug is silent.
- (D) Wrong - memory is identical.

A2. Correct answer: (A) None

- (A) Correct - so the result must be tested before `group()` is called on it.
- (B) Wrong - no Match object is created at all when there is no match.
- (C) Wrong - `search` returns an object or None, never a string.
- (D) Wrong - failure is signalled by the return value, not by an exception.

A3. Correct answer: (A) The entire match

- (A) Correct - `group(0)` and `group()` both mean the whole matched text.
- (B) Wrong - capturing groups are numbered from 1, unusually for Python.
- (C) Wrong - `group(0)` always exists when there is a match.
- (D) Wrong - that is what `groups()` returns.

A4. Correct answer: (A) 'Bat'

- (A) Correct - alternation tries left to right and stops at the first success, truncating the match.
- (B) Wrong - that requires the longer alternative to be listed first.
- (C) Wrong - a match certainly occurs; it is simply shorter than intended.
- (D) Wrong - `group()` returns a string, not a tuple.

A5. Correct answer: (A) None

- (A) Correct - an optional group that did not participate captures None, not an empty string.
- (B) Wrong - assuming an empty string here is exactly what causes `AttributeError` downstream.
- (C) Wrong - that is the value when the optional part is present.
- (D) Wrong - group 1 exists in the pattern, so the index is valid.

A6. Correct answer: (A) `*` allows zero occurrences, `+` requires at least one

- (A) Correct - the difference shows only on input containing none of the repeated element.
- (B) Wrong - both are greedy by default; adding `?` makes either lazy.
- (C) Wrong - this reverses the two.
- (D) Wrong - they differ precisely on the zero case.

A7. Correct answer: (A) `'<to serve><man>'`

- (A) Correct - the dot-star is greedy, so it spans to the last `>` it can.
- (B) Wrong - that is what the nongreedy form `<.*?>` returns.
- (C) Wrong - matching starts at the first `<` in the text.
- (D) Wrong - a match definitely occurs; it is just longer than expected.

A8. Correct answer: (A) Follow it with a question mark

- (A) Correct - `*?` and `+?` match as little as possible.

- (B) Wrong - a caret anchors to the start or negates a class.
- (C) Wrong - square brackets define a character class.
- (D) Wrong - there is no such flag.

A9. Correct answer: (A) A list of tuples, one tuple per match

- (A) Correct - adding groups changes the return shape, which breaks every downstream line written for strings.
- (B) Wrong - that is the shape only when the pattern has no capturing groups.
- (C) Wrong - findall returns plain data, never Match objects.
- (D) Wrong - findall never returns a dictionary.

A10. Correct answer: (A) (?:...)

- (A) Correct - a non-capturing group bundles for a quantifier but keeps findall returning a flat list.
- (B) Wrong - plain parentheses both group and capture.
- (C) Wrong - square brackets define a character class.
- (D) Wrong - curly brackets specify a repetition count.

A11. Correct answer: (A) Any letter, digit or underscore

- (A) Correct - the underscore is included, which surprises people expecting letters only.
- (B) Wrong - that is backslash-s.
- (C) Wrong - it matches a single character, not a whole word.
- (D) Wrong - that is backslash-D.

A12. Correct answer: (A) It negates the character class

- (A) Correct - the caret has a different meaning inside a class than outside one.
- (B) Wrong - that is its meaning outside square brackets.
- (C) Wrong - it is literal only when it is not the first character in the class.
- (D) Wrong - case-insensitivity comes from the re.IGNORECASE flag.

A13. Correct answer: (A) r'^\d+\$'

- (A) Correct - both anchors together require the entire string to match.
- (B) Wrong - unanchored, this matches a run of digits anywhere inside the string.
- (C) Wrong - anchoring only the start still permits trailing rubbish.
- (D) Wrong - the star also matches the empty string, so this accepts anything.

A14. Correct answer: (A) Any single character except a newline

- (A) Correct - re.DOTALL is needed to include the newline.
- (B) Wrong - that is the behaviour only with the DOTALL flag set.
- (C) Wrong - a literal full stop requires escaping, or a character class.
- (D) Wrong - the dot matches exactly one character; a quantifier makes it a sequence.

A15. Correct answer: (A) re.VERBOSE

- (A) Correct - VERBOSE makes whitespace and # comments inside the pattern insignificant.
- (B) Wrong - DOTALL changes what the dot matches.
- (C) Wrong - IGNORECASE makes matching case-blind.
- (D) Wrong - MULTILINE changes what the anchors mean, not how the pattern may be laid out.

A16. Correct answer: (A) join inserts the correct separator for whichever operating system is running

- (A) Correct - a hardcoded separator works on one platform and silently breaks on the other.

- (B) Wrong - speed is irrelevant; both are trivial string operations.
- (C) Wrong - join is pure string work and checks nothing on disk.
- (D) Wrong - concatenation is valid syntax, which is what makes the bug survive to deployment.

A17. Correct answer: (A) It is truncated to nothing immediately

- (A) Correct - 'w' empties the file on opening, which is unrecoverable.
- (B) Wrong - that is 'a' mode.
- (C) Wrong - no exception is raised; the loss is silent.
- (D) Wrong - 'w' is a write mode, and 'r' is the read-only one.

A18. Correct answer: (A) It closes the file even if an exception is raised inside the block

- (A) Correct - a manual close is skipped when an exception unwinds past it, leaving data unflushed.
- (B) Wrong - opening speed is identical.
- (C) Wrong - the mode is still a single argument.
- (D) Wrong - manual open and close is still legal, just less safe.

A19. Correct answer: (A) Values come back as the same Python types, with no parsing needed

- (A) Correct - a stored list returns as a list rather than as a string that must be parsed.
- (B) Wrong - a shelf file is binary and cannot usefully be hand-edited.
- (C) Wrong - the opposite; opacity is the price shelf charges.
- (D) Wrong - shelf must be imported like any other module.

A20. Correct answer: (A) Importing the module must bind a name you can then read

- (A) Correct - importing executes the file, and without a binding there is nothing to reference afterwards.
- (B) Wrong - pformat returns exactly the structure text; the assignment is added by you.
- (C) Wrong - such a file imports perfectly well; it simply exposes nothing.
- (D) Wrong - the assignment makes the file marginally longer.

Part B — model answers and marking scheme

B1 (3 marks) — CO3, L4, Ch.7, pp.148-150

Question. Explain the argument for regular expressions by contrasting them with a hand-written matcher.

Model answer. A hand-written matcher for a fixed format such as 415-555-4242 needs about a dozen explicit checks: the length, each fixed character, and each digit position tested with isdecimal. It works, but the specification is encoded in control flow, so changing the accepted format means rewriting branches rather than editing a string. A regular expression moves the specification into data, so a format change is a one-line pattern change and a second format is a second pattern rather than a second near-identical function. The runtime cost is comparable; the maintenance cost is not.

Point required	Marks
Correct description of the manual approach and its checks	1
Correct identification of brittleness as the real cost	2
Total	3

B2 (3 marks) — CO3, L4, Ch.7, pp.156-157

Question. Explain greedy versus nongreedy matching with a worked example, and say why single-occurrence testing hides the difference.

Model answer. All Python quantifiers are greedy by default, matching as much as possible and backtracking only if forced. Applied to '<to serve><man> for dinner', the pattern `r'<.*>'` matches '<to serve><man>' because the dot-star runs to the end and then gives back only as far as the last >. Adding a question mark makes it nongreedy, so `r'<.*?>'` stops at the first > and matches '<to serve>'. On input containing exactly one tag both patterns return the same thing, which is why a quantified pattern must always be tested against text holding at least two candidates.

Point required	Marks
Correct definition of both behaviours	1
Correct worked example with both outputs	1
Correct explanation of why one occurrence hides the difference	1
Total	3

B3 (3 marks) — CO3, L4, Ch.7, pp.157-158

Question. Explain how the return shape of `findall()` depends on the pattern, and how to keep it flat.

Model answer. With no capturing groups, `findall` returns a list of strings, one per non-overlapping match. With one or more capturing groups it returns a list of tuples, one tuple per match with one element per group. Adding parentheses to an otherwise working pattern therefore changes the type of everything downstream, and code that called a string method on each item raises `AttributeError` on a tuple. Replacing the capturing parentheses with the non-capturing form `(?....)` keeps the grouping for quantifier purposes while restoring the flat list of strings.

Point required	Marks
Both return shapes stated correctly	1
Correct consequence for downstream code	1
Correct non-capturing group solution	1
Total	3

B4 (3 marks) — CO3, L4, Ch.7, pp.159-162

Question. Explain the difference between searching and validating, and write a pattern for each.

Model answer. Searching asks whether the pattern occurs anywhere in the text; validating asks whether the whole text is the pattern. Unanchored, `r'\d+'` matches the 12345 inside 'abc 12345 xyz', so a validator written that way accepts contaminated input. Anchoring both ends with `r'^\d+$'` requires the entire string to be digits, so the same input returns None and only '12345' matches. Forgetting the anchors is why a username validator of `\w+` accepts 'hello world!!' — `\w+` matches 'hello' and the rest is ignored.

Point required	Marks
Correct conceptual distinction	1
Both patterns correct with anchors placed properly	1

Point required	Marks
A concrete example of the unanchored failure	1
Total	3

B5 (2 marks) — CO3, L3, Ch.7, pp.150-152

Question. Why must regular expressions be written as raw strings?

Model answer. Python processes escape sequences in an ordinary string literal before the value is passed anywhere, so '\\d' would be interpreted by Python first and the regex engine would receive something other than what was written. Prefixing with r disables that processing, so the backslashes reach the engine intact. The bug is silent rather than loud, because some sequences survive by accident and others do not, which is why the habit should be unconditional rather than case by case.

Point required	Marks
Correct explanation of the double-processing problem	1
Correct note that the failure is silent and inconsistent	1
Total	2

B6 (2 marks) — CO3, L4, Ch.7, pp.153-154

Question. Explain why alternation order matters, with an example.

Model answer. The engine tries alternatives left to right at each position and stops at the first that succeeds, so it does not look for the longest match. Applied to 'Batman', the pattern r'(Bat|Batman)' matches 'Bat' and stops, leaving 'man' unmatched. Reordering to r'(Batman|Bat)' matches the full 'Batman'. The rule is therefore to list the longest alternative first whenever one alternative is a prefix of another; no error is raised either way, so the symptom is a match that is quietly too short.

Point required	Marks
Correct statement of left-to-right first-success behaviour	1
Correct example with both orderings and their results	1
Total	2

B7 (2 marks) — CO3, L4, Ch.7, pp.154-155

Question. What does group(n) return for an optional group that did not match, and why does it matter?

Model answer. It returns None, not an empty string. For r'Bat(wo)?man' matching 'Batman', mo.group(1) is None, because the group never participated in the match. This matters because any string operation applied to it, such as mo.group(1).upper(), raises AttributeError: 'NoneType' object has no attribute 'upper'. Code reading an optional group must test for None or supply a default before using the value.

Point required	Marks
Correct value identified as None	1
Correct consequence with the exception named	1
Total	2

B8 (2 marks) — CO3, L4, Ch.7, pp.162-165

Question. Explain what `re.VERBOSE` is for and when you would reach for it.

Model answer. `VERBOSE` makes whitespace inside a pattern insignificant and allows `#` comments within it, so a long expression can be laid out over several lines with each part explained. It is worth reaching for as soon as a pattern exceeds about one line, because an unreadable regex cannot be reviewed or corrected later by anyone including its author. Flags are combined with the pipe operator, so `re.VERBOSE | re.IGNORECASE` is the usual form when both are wanted.

Point required	Marks
Correct description of what <code>VERBOSE</code> changes	1
Correct threshold for using it and the flag-combining syntax	1
Total	2

B9 (2 marks) — CO3, L4, Ch.7, pp.162-165

Question. Write a `sub()` call that redacts names after the word `Agent`, keeping only the initial, and explain how the backreference works.

Model answer. Compile `r'Agent (\\w)\\w*'` so that group 1 captures the first letter of the name and the rest is matched but not captured. Then `agentRegex.sub(r'\\1**', text)` **replaces each whole match with the captured letter followed by four asterisks. Applied to 'Agent Alice told Agent Bob' the result is 'A told B**'**. The `\\1` in the replacement string is a backreference to group 1 of that particular match, so each replacement uses its own captured initial, and `sub` returns a new string leaving the original unchanged.

Point required	Marks
Correct pattern and sub call	1
Correct explanation of the backreference and the new-string return	1
Total	2

B10 (2 marks) — CO3, L3, Ch.8, pp.173-177

Question. Explain why `os.path.join` should be used and what goes wrong without it.

Model answer. `os.path.join` inserts whichever separator the running platform uses, producing `'usr\\bin\\spam.txt'` on Windows and `'usr/bin/spam.txt'` elsewhere from one line of code. Building the path by concatenation hardcodes one separator, so the script works on the machine it was written on and breaks silently on the other. The failure surfaces as a file-not-found error far from the line that caused it, which makes it disproportionately expensive to diagnose.

Point required	Marks
Correct description of what <code>join</code> does	1
Correct account of the portability failure	1
Total	2

B11 (2 marks) — CO3, L3, Ch.8, pp.177-180

Question. Write the code to total the sizes of all files in a folder and explain the check it needs.

Model answer. Loop over `os.listdir(folder)`, which returns bare names rather than paths, and join each name to the folder with `os.path.join` to form a usable path. Test each with `os.path.isfile` before calling `os.path.getsize`, because a subdirectory would otherwise contribute its directory-entry size rather than its contents and corrupt the total. Accumulate the sizes into a running total. Passing the bare names from `listdir` straight to `getsize` fails unless the folder happens to be the current working directory.

Point required	Marks
Correct loop with <code>os.path.join</code> applied to each name	1
Correct <code>isfile</code> guard with the reason	1
Total	2

B12 (2 marks) — CO3, L4, Ch.8, pp.180-184

Question. Contrast the three file modes `r`, `w` and `a`, and say which is dangerous and why.

Model answer. `r` opens for reading and fails if the file is absent. `w` opens for writing and truncates the file to nothing the instant it is opened, whether or not anything is subsequently written. `a` opens for appending, so existing content survives and new data is added at the end. `w` is the dangerous one because the truncation is immediate, silent and unrecoverable; reopening a file in '`w`' inside a loop intending to add to it leaves only the final write. Append mode is what adds.

Point required	Marks
All three modes described correctly	1
Correct identification of <code>w</code> as dangerous with the loop example	1
Total	2

B13 (2 marks) — CO3, L5, Ch.8, pp.184-192

Question. Compare `shelve` with `pprint.pformat` as ways of persisting a Python structure.

Model answer. `shelve` writes a binary file and returns values with their Python types intact, so a stored list comes back a list with no parsing; it handles a wide range of types but the file is opaque and cannot be hand-edited. `pformat` returns the structure as valid Python source, which can be written to a `.py` file with a leading assignment and imported back, giving persistence that a person can open, read and correct in any editor. Its limits are that it suits basic types only and that importing executes the file, so it must be trusted. Choose by whether a human needs to read the saved data.

Point required	Marks
Correct description of both mechanisms	1
Correct statement of the trade-off and the selection criterion	1
Total	2